Machine Learning Practical No. 1

Feature Engineering: Missing Values, Encoding and Scaling

Aim: Apply basic feature engineering techniques such as handling missing values, encoding categorical variables, and scaling numerical attributes on the Titanic dataset to prepare a clean, model-ready dataset.

Theory / Key Concepts

Feature Engineering: Transforming raw data into useful features suitable for machine learning.

Missing-value handling: Replacing unavailable values using a suitable strategy such as mean or most frequent value.

Encoding: Converting categorical/text values into numerical form.

Scaling: Standardizing numerical features so they are on a similar scale.

Step-by-Step Procedure

Step 1 — Open Anaconda Navigator: Start Menu → Anaconda Navigator.

Step 2 — Launch Jupyter Notebook: Click Launch under Jupyter Notebook.

Step 3 — Create Notebook: New → Python 3. Rename it Feature_Engineering.ipynb.

Step 4 — Import Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

Step 5 — Load Dataset

```
url="https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df=pd.read_csv(url)
```

Step 6 — Display Dataset

```
df.head()
```

Step 7 — Display Dataset Information

```
df.info()
```

Step 8 — Check Missing Values

```
df.isnull().sum()
```

Step 9 — Select Required Columns

The practical selects columns needed to demonstrate numerical processing, categorical encoding, and a target variable.

```
df=df[['Pclass',
       'Sex',
       'Age',
       'SibSp',
       'Parch',
       'Fare',
       'Embarked',
       'Survived']]

df.head()
```

Step 10 — Separate Input and Output

```
X=df.drop('Survived',axis=1)
y=df['Survived']
```

X = input/features. **y** = target/output.

Step 11 — Identify Numerical Features

```
numeric_features=['Age',
                  'Fare',
                  'SibSp',
                  'Parch']
```

Step 12 — Identify Categorical Features

```
categorical_features=['Sex',
                      'Embarked',
                      'Pclass']
```

Step 13 — Numerical Preprocessing

```
numeric_transformer=Pipeline(steps=[
    ('imputer',SimpleImputer(strategy='mean')),
    ('scaler',StandardScaler())
])
```

Missing numerical values are replaced with the mean, then numerical values are standardized.

Step 14 — Categorical Preprocessing

```
categorical_transformer=Pipeline(steps=[
    ('imputer',SimpleImputer(strategy='most_frequent')),
    ('encoder',OneHotEncoder(handle_unknown='ignore'))
])
```

Missing categorical values are replaced with the most frequent value, then categories are one-hot encoded.

Step 15 — Combine Transformations

```
preprocessor=ColumnTransformer(
    transformers=[
        ('num',numeric_transformer,numeric_features),
        ('cat',categorical_transformer,categorical_features)
    ])
```

Step 16 — Split Dataset

```
X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42)
```

80% is used for training and 20% for testing.

Step 17 — Apply Feature Engineering

```
X_train_processed=preprocessor.fit_transform(X_train)
X_test_processed=preprocessor.transform(X_test)
```

Remember: Training data uses **fit_transform()**; test data uses **transform()**.

Step 18 — Display Dataset Shape

```
print("Original Shape :",X_train.shape)
print("Processed Shape :",X_train_processed.shape)
```

The number of columns can increase because One-Hot Encoding creates additional binary columns.

Step 19 — Success Message

```
print("Feature Engineering Completed Successfully")
```

Complete Program

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

url="https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df=pd.read_csv(url)

df.head()
df.info()
df.isnull().sum()

df=df[['Pclass','Sex','Age','SibSp','Parch','Fare','Embarked','Survived']]
df.head()

X=df.drop('Survived',axis=1)
y=df['Survived']

numeric_features=['Age','Fare','SibSp','Parch']
categorical_features=['Sex','Embarked','Pclass']

numeric_transformer=Pipeline(steps=[
    ('imputer',SimpleImputer(strategy='mean')),
    ('scaler',StandardScaler())
])

categorical_transformer=Pipeline(steps=[
    ('imputer',SimpleImputer(strategy='most_frequent')),
    ('encoder',OneHotEncoder(handle_unknown='ignore'))
])

preprocessor=ColumnTransformer(transformers=[
    ('num',numeric_transformer,numeric_features),
    ('cat',categorical_transformer,categorical_features)
])

X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=0.20,random_state=42)

X_train_processed=preprocessor.fit_transform(X_train)
X_test_processed=preprocessor.transform(X_test)

print("Original Shape :",X_train.shape)
print("Processed Shape :",X_train_processed.shape)
print("Feature Engineering Completed Successfully")
```

Quick Viva Revision

What is feature engineering? Transforming raw data into useful machine-learning features.

What is X? Input features. **What is y?** Target/output.

Numerical pipeline? Mean imputation + StandardScaler.

Categorical pipeline? Most-frequent imputation + OneHotEncoder.

Why ColumnTransformer? To apply different preprocessing to different columns.

Why split? To train on one part and test on unseen data.

fit_transform vs transform? fit_transform learns and applies preprocessing; transform applies learned preprocessing.

Why do processed columns increase? One-Hot Encoding creates extra columns.

Exam Memory Trick

IMPORT → **LOAD** → **CHECK** → **X/y** → **NUM/CAT** → **PIPELINES** → **COMBINE** → **SPLIT** → **TRANSFORM** → **OUTPUT**